**CIÊNCIAS DA COMPUTAÇÃO**

Beatriz Rodrigues de Souza - RA: R019640

Maria Eduarda Santos Correia - RA: R015IE0

# DESENVOLVIMENTO DE JOGOS A PARTIR DA APLICAÇÃO DE PRIMITIVAS GRÁFICAS.

SÃO PAULO

2026

# Sumário

1.Descrição do jogo ..... 3
2. Descrição com prints ..... 4
2.1 Início e Interface Principal ..... 4
2.2 Mecânicas de Movimento e Obstáculos ..... 4
2.3 Sistema de Combate e Divisão ..... 5
2.4 Eventos Temporais (A Centopeia se Divide) ..... 6
2.5 Condições de Finalização ..... 6

Figura 1: Inicio da Tela ..... 4
Figura 2: disparos com as teclas A e D (na horizontal) ..... 5
Figura 3: Tiros com a tecla espaço(vertical) ..... 5
Figura 4: Com a adição de 10s extras. ..... 6
Figura 5: Tela de Vitória ..... 6
Figura 6:Tela de Gameover ..... 7
Figura 7:Vetores Pygame ..... 7
Figura 8:Algortimos Vetoriais ..... 7
Figura 9: movimentação de zigue zague ..... 8
Figura 10: Fundo com o Plano cartesiano 40x40x40 ..... 8

# 1.Descrição do jogo

O jogo desenvolvido é uma releitura do clássico arcade *Centipede*, fundamentada na aplicação prática de conceitos de Computação Gráfica e Desenvolvimento de Jogos através da biblioteca Pygame.
Toda a parte visual foi construída utilizando primitivas gráficas, que são as formas geométricas básicas desenhadas em tempo real pelo código. Em vez de usar imagens prontas (bitmaps), trabalhei na camada mais pura da renderização, o que garante total controle sobre os objetos.

O personagem principal, o gnomo, é um triângulo gerado por polígonos; a centopeia e os projéteis são círculos; e os cogumelos combinam círculos e retângulos. Essa técnica aplica o conceito de rasterização, onde o computador converte descrições geométricas em pixels na tela, permitindo criar elementos visuais complexos a partir de formas simples.

A lógica de movimentação e a organização do cenário baseiam-se no sistema de coordenadas e no plano cartesiano. Na computação gráfica, a tela funciona como um mapa 2D onde o eixo X cresce para a direita e o eixo Y cresce para baixo. Para tornar essa estrutura matemática visível e facilitar a percepção espacial, implementei uma grade de fundo composta por linhas, que serve como uma malha de referência. Sem esse sistema, seria impossível posicionar ou alinhar o gnomo e os inimigos com precisão dentro do espaço digital.

Para fazer tudo se mover, utilizei a Álgebra Linear, trabalhando com pontos e vetores. No código, o ponto define a localização exata de cada objeto, enquanto o vetor define para onde ele vai e com que intensidade (velocidade). Por exemplo, a centopeia realiza movimentos de translação horizontal até detectar as bordas ou obstáculos, momento em que o código inverte sua direção e aplica um incremento no eixo vertical, criando o movimento em zigue-zague. Quando o gnomo dispara, usamos direções vetoriais para guiar o projétil pelo espaço sem a necessidade de cálculos de ângulos manuais complexos a cada instante.

A interatividade do jogo depende diretamente da detecção de colisões, onde apliquei o conceito de distância euclidiana. Para checar se um tiro atingiu a centopeia, o sistema utiliza o Teorema de Pitágoras para calcular a distância em linha reta entre os centros das formas. Se essa distância for menor que o raio dos círculos, ocorre a colisão.

Essa escolha foi feita por ser muito mais precisa para objetos arredondados do que o uso de "caixas" quadradas, evitando que o jogador sinta que acertou o vazio.
Por fim, o projeto demonstra o funcionamento do Pipeline Gráfico: definimos os dados (pontos e vetores), aplicamos transformações (movimentação), realizamos os testes de colisão e, por fim, renderizamos as primitivas na tela. O resultado é um sistema que une matemática vetorial e geometria analítica para transformar cálculos abstratos em uma experiência de jogo dinâmica, responsiva e visualmente clara.

## 2. Descrição com prints

O projeto consiste em um jogo de tiro e sobrevivência inspirado no clássico *Centipede*. O jogador assume o controle de um "Gnomo" (triângulo azul) e deve enfrentar uma centopeia que desce pela tela. Abaixo, detalho as etapas e mecânicas visuais:

### 2.1 Início e Interface Principal

Ao iniciar o jogo, o cenário é apresentado com uma grade de fundo (mesh), que serve como guia visual para o plano cartesiano. No topo, há um cronômetro que define o tempo de sobrevivência ou o tempo para eliminar a ameaça.

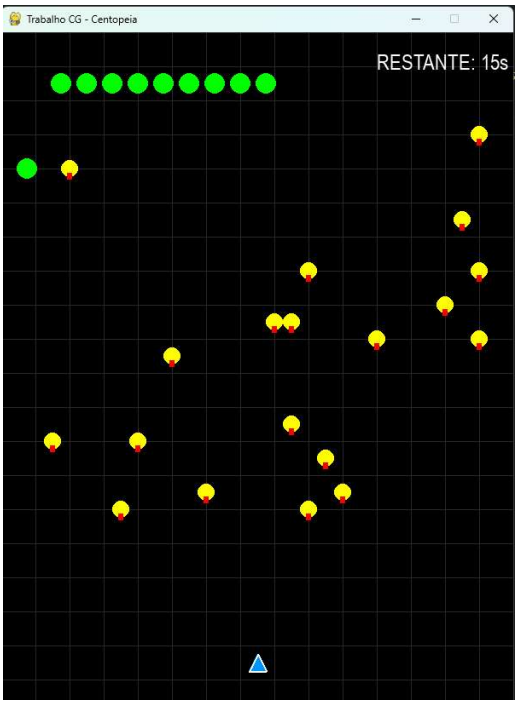


Figura 1: Inicio da Tela

### 2.2 Mecânicas de Movimento e Obstáculos

O jogador pode se mover livremente em todas as direções na parte inferior da tela. O cenário é repleto de cogumelos (círculos amarelos com base vermelha) gerados aleatoriamente.

- Influência no Inimigo: Quando a centopeia atinge a borda da tela ou colide com um desses cogumelos, ela inverte sua direção horizontal e desce um nível no eixo Y. Isso torna a trajetória do inimigo imprevisível.

## 2.3 Sistema de Combate e Divisão

O jogador possui três tipos de disparos (Cima, Esquerda e Direita), permitindo uma jogabilidade mais dinâmica.

- Atingindo o Inimigo: Ao disparar contra a centopeia, cada segmento atingido é removido. Na lógica do código, os segmentos posteriores ao que foi atingido mudam de direção, simulando a quebra do corpo do inimigo.

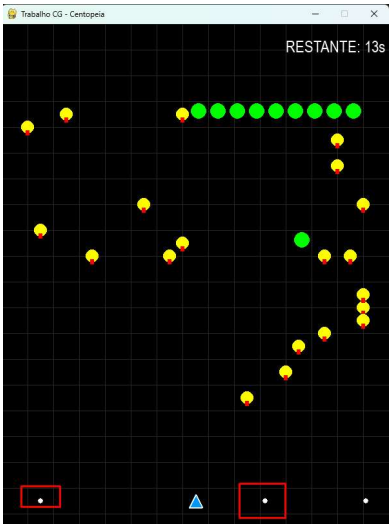


Figura 2: disparos com as teclas A e D (na horizontal)

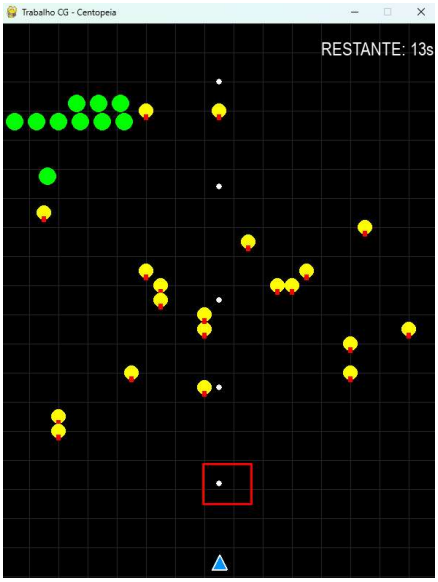


Figura 3: Tiros com a tecla espaço(vertical)

## 2.4 Eventos Temporais (A Centopeia se Divide)

Se o tempo limite de 15 segundos se esgotar e o jogador não tiver eliminado a ameaça, o jogo ativa um evento especial: uma nova centopeia surge (ou o inimigo se divide), e o jogador recebe um bônus de "10s EXTRAS" para tentar limpar a tela.

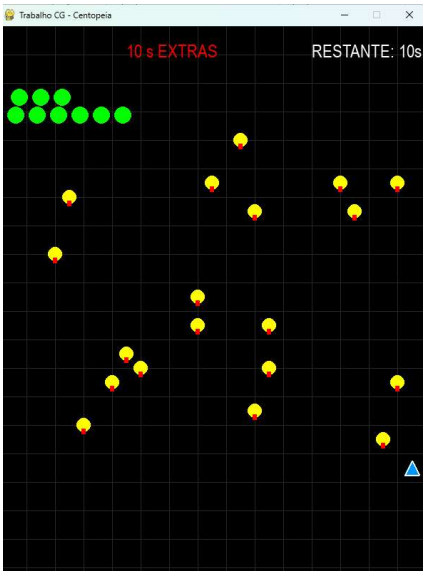


Figura 4: Com a adição de 10s extras.

## 2.5 Condições de Finalização

O jogo possui dois estados finais claros baseados na detecção de colisão por distância euclidiana:

- Vitória: Ocorre quando todos os segmentos da centopeia são removidos. O jogo exibe o tempo total gasto.

Figura 5: Tela de Vitória

- Game Over: Ocorre se qualquer segmento da centopeia tocar o triângulo do jogador ou se o tempo extra se esgotar.

Figura 6:Tela de Gameover

```
for seg in self.corpo[:]:

    if t.pos.distance_to(seg.pos) < 20: # Cálculo de colisão usando distância vetorial
```

```
class Gnomo:
    def __init__(self):
        self.pos = pygame.Vector2(LARGURA // 2, ALTURA - 60) # Definição da posição como um vetor
```

Figura 7:Vetores Pygame

- O uso da classe **pygame.Vector2** é o que permite que o jogo saia de simples coordenadas estáticas para um sistema dinâmico de movimento.

```
for seg in self.corpo[:]:

    if t.pos.distance_to(seg.pos) < 20: # Cálculo de colisão usando distância vetorial
```

Figura 8:Algortimos Vetoriais

- Toda a interface é desenhada via algoritmos de rasterização de formas básicas. Ou seja, ele converte formas geométricas vetoriais (pontos, linhas, círculos, polígonos) em pixels. Com a apalicação de draw.polygon, draw.circle e draw.rect.

```python
for seg in self.corpo:
    seg.mover()

    colidiu_cogumelo = False
    for cog in self.cogumelos:

        if seg.pos.distance_to(cog) < 20:
            colidiu_cogumelo = True
            break

    if seg.pos.x > LARGURA - 15 or seg.pos.x < 15 or colidiu_cogumelo:
        seg.direcao *= -1
        seg.pos.y += 25
        seg.pos.x += seg.direcao * seg.velocidade
```

Figura 9: movimentação de zigue zague

- A movimentação da centopeia é um exemplo prático de transformações geométricas (translação e reflexão) aplicadas via código. A lógica de zigue-zague utiliza o sinal da variável direcao para controlar o sentido do vetor velocidade. Quando uma condição de contorno é atingida, o código executa duas transformações: uma **reflexão** horizontal e **translação** vertical fixa.

```python
def desenhar_grade():
    espacamento = 40
    for x in range(0, LARGURA, espacamento):
        pygame.draw.line(tela, (40,40,40), (x,0), (x,ALTURA))
```

Figura 10: Fundo com o Plano cartesiano 40x40x40

- A função desenhar_grade não é apenas estética; ela define visualmente o **Espaço do Mundo** do jogo. Ao desenhar linhas a cada 40 pixels, o código materializa o **Plano Cartesiano**. Isso ajuda a validar se as transformações espaciais e as detecções de colisão estão ocorrendo nas coordenadas corretas.